RETRIEVAL-AUGMENTED GENERATION (RAG)

Name: _____ **Branch/Year:** _____ **Date:** _____

Warm-Up: Prompt Engineering & Fine-Tuning Recall

1. If you want an LLM to accurately answer questions based on your company's private PDF manuals, what should you do?

A. Write a highly detailed System Prompt

B. Fine-tune the model with LoRA on the PDFs

C. Create a system that feeds text from the PDFs into the prompt at runtime

D. Reduce the temperature parameter to 0

2. Fine-Tuning is best used when you want the AI to:

A. Adopt a specific format, tone, or style permanently

B. Search Wikipedia for live factual information

C. Answer questions about yesterday's stock prices

D. Access your company's private database

3. What is a "hallucination" in AI?

A. When the model crashes due to memory overflow

B. The AI generates confident-sounding but factually incorrect information

C. A visual glitch in the AI's interface

D. When the AI refuses to answer a question

Part 1: The Problem with LLMs — Why Isn't GPT-4 Enough?

THE SCENARIO: "The Brilliant But Stuck-In-Time Assistant"

Imagine you've hired a genius assistant who has read every book, every website, and every paper ever published — but only up to January 2024. Since that day, they've been locked in a room with no internet, no news, and no access to your company's files.

You ask: "*What were yesterday's top headlines?*" — They don't know.
You ask: "*What does our Employee Handbook say about sick leave?*" — They make something up.
You ask: "*Where did you get that answer?*" — They can't tell you.

This is **exactly how LLMs work today.** They're incredibly smart, but they have critical blind spots.

Large Language Models (like GPT-4, LLaMA, Gemini) are Transformers trained on massive datasets to predict the next token. They're brilliant at language but have **four major limitations:**

Limitation	What It Means	Example
Knowledge Cutoff	They are "frozen in time" — they only know what existed on the internet up to their training date.	Can't tell you who won the cricket match yesterday.
Hallucinations	When they don't know the answer, they *confidently make up facts* instead of saying "I don't know."	Invents a citation to a research paper that doesn't exist.
No Private Data	They have zero access to your personal files, company databases, or internal documents.	Can't answer: "What is our company's refund policy?"
No Sources	They can't tell you *where* an answer came from — you can't verify or cite it.	Claims "according to the 2024 report" but can't point to the actual report.

ACTIVITY: Think & Discuss:

If you ask ChatGPT: **"What is my bank account balance?"** — why can't it answer?

Write two reasons:
1.
2.

KEY INSIGHT: LLMs are **not fact databases**. They are pattern-matching machines that predict the next likely token. When they "know" something, it's because the pattern appeared in their training data — not because they looked it up.

Part 2: What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that solves all four limitations we just discussed. Instead of relying on the AI's internal "memory," we **feed it external facts** right before it answers.

In Simple Words: RAG = AI + Your Private Data. The AI reads your documents first, then answers your question based *only* on what it just read.

THE ANALOGY: Memory Exam vs. Open-Book Exam

Standard LLM (No RAG): A student taking a test *from memory*.
They might remember the concepts, but details get fuzzy. Sometimes they make up answers to fill gaps.

LLM + RAG: A student taking an *open-book exam*.
They look up the relevant page in the textbook **first**, then write a precise answer using that information. Much more accurate!

OBSERVATION: Notice the key difference: in Prompt Engineering and Fine-Tuning, we rely on what the AI already has in its "brain." In RAG, we treat the AI as a **reasoning engine** — we give it the facts, and it writes a great answer from those facts.

ACTIVITY: Fill in the Blanks:

RAG stands for - - .

The analogy for RAG is: an exam.

> In RAG, the AI reads external _____ before generating its answer.
>
> Without RAG, LLMs rely only on their internal _____ .

Quick Check: What is the core idea behind RAG?

A. Retrain the model's weights with new data

B. Provide external documents in the prompt so the AI answers from real facts

C. Increase the temperature to get more creative answers

D. Use a bigger model with more parameters

Part 3: The RAG Workflow — Step by Step

Here's how RAG works, step by step. Every RAG system follows this exact pipeline:

USER QUESTION

"What is the leave policy?"

YOUR DOCUMENTS

Employee Handbook.pdf
HR Policy 2024.docx

STEP 1: RETRIEVAL (SEARCH)

The system searches the documents and finds the **perfect paragraph** answering the user's question.

STEP 2: AUGMENTATION (COMBINE)

The model **merges** the original question + the retrieved text into a single, super-informative prompt.

STEP 3: GENERATION (ANSWER)

The LLM reads the combined prompt and writes an **accurate, grounded answer**.

FINAL OUTPUT

"*According to the Employee Handbook Section 4.2, employees are entitled to 18 days of leave...*"

Breaking Down Each Step:

Step	What Happens	Example
1. User Query	The user asks a question in plain language.	"What is the leave policy?"
2. Retrieval	The system searches a database for the most relevant document chunks.	Finds: "Section 4.2 — Leave Policy: Employees get 18 days earned leave..."
3. Augmentation	The retrieved text is combined with the question into a prompt.	"Context: [Leave Policy Text]. Question: What is the leave policy? Answer:"
4. Generation	The LLM reads the augmented prompt and generates an answer grounded in facts.	"Employees are entitled to 18 days of earned leave per calendar year."

ACTIVITY: Trace the Pipeline:

A hospital wants to build a chatbot that answers patient questions using their medical FAQs document. Fill in each step:

User Query: "Can I eat before my blood test?"

Retrieval: The system searches and finds:

Augmentation: The prompt becomes: "Context: . Question: Can I eat before my blood test? Answer:"

Generation: The AI responds:

Part 4: Embeddings — How Computers Understand Meaning

The magic of RAG's retrieval step is **embeddings** — a way to convert text into numbers that capture *meaning*, not just characters.

In Simple Words: Imagine every word in the English language has a secret GPS coordinate. Words with similar meanings (like "happy" and "joyful") have coordinates that are very close together. Words with different meanings (like "happy" and "earthquake") are far apart. That's what embeddings do — they give every piece of text a "location" in a mathematical space.

The Famous Example:

The Word Arithmetic Trick

"King" − "Man" + "Woman" ≈ "Queen"

This actually works with embeddings! The AI captures relationships like gender, royalty, and number as directions in its mathematical space. This is how it "understands" meaning.

Code: Generating Embeddings

What's happening below: We use a free library called sentence-transformers to convert sentences into vectors (lists of numbers). Sentences with similar meanings will produce similar vectors.

```
 # Install: pip install sentence-transformers
from sentence_transformers import SentenceTransformer

# Load a free, pre-trained embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")


# Convert sentences into vectors (embeddings)
sentences = [
    "The cat sat on the mat",
    "A kitten was sitting on the rug",       # Similar meaning!
    "The stock market crashed yesterday"      # Completely different
]

embeddings = model.encode(sentences)


# Each sentence is now a list of 384 numbers
print(f"Shape: {embeddings.shape}")      # (3, 384) → 3 sentences, 384 dimensions each
print(f"First 5 numbers of sentence 1: {embeddings[0][:5]}")
```

OBSERVATION: Notice that we didn't teach the model anything! The embedding model was pre-trained to understand meaning. All we did was pass in sentences, and it returned mathematical representations of their meaning.

ACTIVITY: Predict the Similarity:

Which pairs of sentences would have embeddings that are CLOSE together (similar vectors)? Write **Close** or **Far**:

1. "I love pizza" vs. "Pizza is my favorite food" → _____

2. "I love pizza" vs. "The economy is in recession" → _____

3. "How do I reset my password?" vs. "I forgot my login credentials" → _____

4. "How do I reset my password?" vs. "What is the weather today?" → _____

Part 5: Semantic Search vs. Keyword Search

This is where embeddings become truly powerful. Traditional search (like Google circa 2005) works by matching **exact keywords**. Semantic search (using embeddings) matches **meaning**.

Feature	Keyword Search	Semantic Search (Vectors)
How It Works	Looks for exact word matches	Compares the *meaning* of queries using embeddings
Search: "Car"	Only finds documents containing the word "Car"	Also finds "Automobile", "Vehicle", "SUV" — because they *mean* similar things
Search: "How to fix errors"	Misses "Debugging techniques" (different words)	Finds "Debugging techniques" — same meaning, different words
Technology	Simple string matching (SQL LIKE, regex)	Vector similarity (Cosine Similarity)

How Cosine Similarity Works

In Simple Words: Cosine similarity measures the **angle** between two vectors. If two vectors point in the same direction (similar meaning), the cosine is close to **1**. If they point in completely different directions (unrelated), the cosine is close to **0**.

Code: Measuring Similarity Between Sentences

What's happening below: We compute embeddings for three sentences and measure how similar they are to each other using cosine similarity.

```
 # pip install sentence-transformers scikit-learn
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer("all-MiniLM-L6-v2")


# Three sentences — two are similar, one is different
sentences = [
    "How do I return a product?",            # Sentence A
    "What is the refund policy?",            # Sentence B (similar to A)
    "What time does the store open?"         # Sentence C (different topic)
]

# Convert to embeddings
embeddings = model.encode(sentences)


# Calculate how similar each pair is (1 = identical, 0 = unrelated)
similarity_matrix = cosine_similarity(embeddings)

print("Similarity between A and B:", round(similarity_matrix[0][1], 3))  # ~0.75 (high!)
print("Similarity between A and C:", round(similarity_matrix[0][2], 3))  # ~0.15 (low!)
```

> **KEY INSIGHT:** This is the heart of RAG's retrieval step! When a user asks a question, the system converts it to an embedding and finds the **most similar document chunks** using cosine similarity. It doesn't need exact keyword matches — it understands meaning.

Quick Check: A user searches for "headache medicine." Which would a semantic search find that a keyword search would miss?

A. "Headache medicine for adults"

B. "Pain relief tablets and analgesics"

C. "Headache causes and prevention"

D. "Buy headache medicine online"

Part 6: Vector Databases — Where Embeddings Live

Regular databases (like MySQL or PostgreSQL) store rows and columns. They're great for exact lookups: "*Find all users where age = 25.*"

But for semantic search, we need to store **embeddings (vectors)** and find the **nearest neighbors** by meaning. That's what a **Vector Database** does.

In Simple Words: A vector database is like a library where books are arranged by topic similarity instead of alphabetical order. When you describe what you want, the librarian instantly pulls the most relevant books — even if you didn't know the exact title.

Regular Database (SQL)	Vector Database
Stores text, numbers, dates	Stores embeddings (lists of numbers)
Searches by exact match or filters	Searches by meaning similarity
Query: `WHERE title = 'Car'`	Query: "Find documents about vehicles" (finds Car, SUV, Automobile)
Examples: MySQL, PostgreSQL	Examples: ChromaDB, Pinecone, FAISS, Weaviate

Code: Store & Search Documents with ChromaDB

What's happening below: We create a vector database using ChromaDB (free, runs locally), add some company documents to it, then search by meaning.

```
 # pip install chromadb
import chromadb

# Create a local vector database (no server needed!)
client = chromadb.Client()

# Create a "collection" — like a table in SQL
collection = client.create_collection(name="company_docs")


# Add documents to the collection
# ChromaDB automatically converts text to embeddings!
collection.add(
    documents=[
        "Employees get 18 days of earned leave per year.",
        "The office dress code is business casual on weekdays.",
        "Remote work is allowed on Fridays with manager approval.",
        "Health insurance covers the employee and up to 2 dependents.",
        "Annual performance reviews happen in December."
    ],
    ids=["doc1", "doc2", "doc3", "doc4", "doc5"]    # Unique ID for each doc
)


# Now search by MEANING — not by keywords!
results = collection.query(
    query_texts=["How many vacation days do I get?"],    # User's question
    n_results=2                                           # Return top 2 matches
)

# The system finds "earned leave" even though we searched for "vacation days"!
print("Most relevant documents:")
for doc in results["documents"][0]:
    print(f"  → {doc}")
```

OBSERVATION: Notice that we searched for "vacation days" but the system found the document about "earned leave." A keyword search would have missed this completely! This is the power of semantic search with vector databases.

ACTIVITY: Design Your Collection:

Imagine you're building a RAG chatbot for a **university**. List 4 documents you would add to the vector database:

YOUR DOCUMENTS:

Part 7: Building a Complete RAG Pipeline

Now let's put it all together! Here's a complete, working RAG system that:

1. Loads your documents
2. Converts them to embeddings
3. Stores them in a vector database
4. Searches for relevant context
5. Sends the context + question to an LLM
6. Returns a grounded, accurate answer

Step 1: Setup — Load Documents & Build the Vector Index

What's happening below: We load our company documents, split them into smaller chunks (because LLMs have token limits), convert them into embeddings, and store everything in ChromaDB.

```
 # pip install chromadb sentence-transformers openai
import chromadb
from sentence_transformers import SentenceTransformer

# Our "company documents" (in real life, you'd load these from PDF/DOCX files)
documents = [
    "Employees are entitled to 18 days of earned leave per calendar year. "
    "Unused leave can be carried forward up to a maximum of 30 days.",

    "The company provides health insurance covering the employee, spouse, "
    "and up to two children. Dental and vision are covered under the premium plan.",

    "Remote work is permitted on Fridays with prior manager approval. "
    "Employees must be available on Slack between 10 AM and 4 PM.",

    "Performance reviews are conducted annually in December. "
    "Ratings are on a 5-point scale: Exceptional, Exceeds, Meets, Below, Unsatisfactory.",

    "The company reimburses up to Rs. 50,000 per year for professional "
    "certifications and approved training programs."
]


# Create the embedding model and the vector database
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
client = chromadb.Client()
collection = client.create_collection("hr_knowledge_base")


# Convert documents to embeddings and store them
embeddings = embedding_model.encode(documents).tolist()

collection.add(
    documents=documents,
    embeddings=embeddings,
    ids=[f"doc_{i}" for i in range(len(documents))]
)

print(f"Loaded {len(documents)} documents into the vector database!")
```

Step 2: Query — Search & Generate an Answer

What's happening below: The user asks a question. We search the vector database for the most relevant documents, combine them with the question into a prompt, and send it to an LLM for a grounded answer.

```
import os
from openai import OpenAI

# Connect to OpenAI (or use Groq, Ollama, etc.)
llm_client = OpenAI(api_key=os.getenv("OPENAI_KEY"))


def ask_rag(question):
    """The complete RAG pipeline in one function."""

    # STEP 1: RETRIEVE — Find relevant documents
    query_embedding = embedding_model.encode([question]).tolist()

    results = collection.query(
        query_embeddings=query_embedding,
        n_results=2                          # Get top 2 most relevant chunks
    )

    context = "\n".join(results["documents"][0])


    # STEP 2: AUGMENT — Build the prompt with context
    augmented_prompt = f"""You are a helpful HR assistant. Answer the question
using ONLY the context provided below. If the answer is not in the context,
say "I don't have that information."

Context:
{context}

Question: {question}
Answer:"""


    # STEP 3: GENERATE — Send to the LLM
    response = llm_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": augmented_prompt}],
        temperature=0.3           # Low temperature = more factual
    )

    return response.choices[0].message.content


# Try it!
answer = ask_rag("How many leave days do I get per year?")
print(f"Answer: {answer}")
```

KEY INSIGHT: This is a production-grade RAG pattern! The critical line is the augmented prompt — notice how we tell the LLM to answer **ONLY from the context**. This prevents hallucinations because the AI won't make up facts; it's restricted to what we gave it.

ACTIVITY: Modify the Pipeline:

The current system uses an HR knowledge base. How would you change it to build a RAG chatbot for a **restaurant menu**?

1. What documents would you load?

2. What would you change in the system prompt?

3. Write an example user question:

Part 8: RAG vs. Fine-Tuning — Choosing the Right Tool

Now that you've learned both approaches, here is the critical decision framework: **RAG is for Knowledge; Fine-Tuning is for Behavior.**

Dimension	RAG	Fine-Tuning
Best For	Facts, private data, dynamic/live information	Style, tone, format, new skills
How It Works	AI reads external documents before answering	AI's internal weights are permanently retrained
Analogy	Open-book exam	Medical school (permanent learning)
Data Changes?	Just update the documents — no retraining!	Must retrain the entire model again
Hallucination Fix?	**Yes!** Grounds answers in real documents	**No.** Can make it worse with bad data
Source Citations?	**Yes!** Can tell you which document the answer came from	No. Knowledge is baked into the weights
Cost	Cheaper (no GPU needed for training)	Expensive (GPU time + retraining)
Use Case Examples	Company wiki, daily news, legal docs, medical FAQs	Brand voice, medical terminology, language style

KEY INSIGHT: The best production AI systems combine both: **Fine-Tune** the model for instruction following and style + **RAG** for factual content. This is the state-of-the-art in industry.

ACTIVITY: RAG or Fine-Tune?

For each scenario, write **RAG** or **FT** (Fine-Tuning):

1. A chatbot that answers questions about today's weather forecast → _____

2. A model that always replies in formal British English → _____

3. A legal assistant that searches through 10,000 contracts → _____

4. A customer service bot with a warm, empathetic personality → _____

5. A healthcare bot that references the latest drug interaction database → _____

6. Making a small 8B model as good as GPT-4 at writing SQL → _____

Part 9: Real-World Applications & Benefits

Where Is RAG Used in the Real World?

Industry	Application	What Gets Retrieved
Customer Support	AI chatbots that answer product questions	FAQ documents, product manuals, troubleshooting guides
Legal	Contract analysis and clause extraction	Legal contracts, case law databases, regulatory docs
Healthcare	Medical literature search and diagnosis support	Research papers, drug databases, clinical guidelines
Education	AI tutors that answer from course materials	Textbooks, lecture notes, syllabi
Enterprise	Internal knowledge base chatbots	Employee handbooks, Slack messages, internal wikis

Key Benefits of RAG

- **Accuracy:** Answers grounded in real documents, not AI guesses.
- **Trustworthy:** You can cite exactly which document the answer came from.
- **Cost-Effective:** No expensive GPU training — just update your document database.
- **Always Up-to-Date:** Add new documents anytime without retraining the model.
- **Privacy-Friendly:** Documents stay on your servers — never sent to train the AI.

Quick Check: A hospital wants an AI that helps doctors look up the latest drug interaction warnings. Why is RAG the best approach?

A. Because drug data never changes

B. Because RAG can provide citations and the data updates frequently

C. Because fine-tuning is free

D. Because RAG changes the model's internal weights

Part 10: Limitations of RAG

RAG is powerful, but it's **not perfect**. Understanding its limitations helps you build better systems.

Limitation	Explanation	How to Handle It
Retrieval Quality	If the retrieval step pulls the wrong documents, the answer will be wrong too. Garbage in = garbage out.	Use better embeddings, tune chunk sizes, add metadata filters.
Chunk Size Tuning	Documents are split into chunks. If chunks are too small, they lack context. Too big, they add noise.	Experiment with chunk sizes (200-500 tokens is a good start).
Context Window Limits	LLMs have a maximum input size. If you retrieve too many documents, they won't all fit in the prompt.	Only retrieve the top 2-5 most relevant chunks.
Latency	The retrieval step adds time. Embedding the query + searching the database takes extra milliseconds.	Use fast vector databases (FAISS) and cache frequent queries.
Complex Reasoning	RAG struggles when the answer requires combining facts from many different documents.	Use multi-step retrieval or chain-of-thought prompting.

WARNING: **RAG does NOT make the AI smarter.** It makes the AI better *informed*. If the retrieved documents are wrong, outdated, or irrelevant, the AI will still give a bad answer — just one that sounds more convincing because it cites a source.

ACTIVITY: True or False:

1. RAG guarantees 100% accurate answers. _____

2. Chunk size affects retrieval quality. _____

3. RAG can answer questions that require no external data. _____

4. Vector databases are faster at semantic search than SQL databases. _____

Part 11: Key Terminology

ACTIVITY: Terminology Matching:

Match each term to its definition (draw lines or write letters next to numbers):

1. Embedding
2. Vector Database
3. Cosine Similarity
4. Retrieval
5. Augmentation
6. Hallucination
7. Chunk
8. Knowledge Cutoff

A. The date after which the model has no training data
B. A smaller piece of a large document used for indexing
C. Combining the user question with retrieved context into one prompt
D. When AI confidently makes up false information
E. A mathematical measurement of how similar two vectors are
F. A list of numbers representing the meaning of text
G. Searching a database for the most relevant documents
H. A database designed to store and search embeddings by similarity

1:__ 2:__ 3:__ 4:__ 5:__ 6:__ 7:__ 8:__

Part 12: Final Challenge — The AI Architect

THE GRAND CHALLENGE: Enterprise Knowledge Base Chatbot

A **large e-commerce company** wants to build an internal AI assistant for their customer support team. The system must:

- Answer questions about 500+ product pages
- Reference the company's return and refund policies
- Know about current promotions (which change weekly)
- Cite the exact source document for every answer
- Never make up product features that don't exist

Using everything you learned today, design their system:

1. Should they use RAG, Fine-Tuning, or Both? Justify your answer:

   YOUR STRATEGY & JUSTIFICATION:

2. List 4 types of documents they should load into the vector database:

   DOCUMENTS FOR THE VECTOR DB:

3. Why can't they just fine-tune GPT-4 with all their product data? Give 2 reasons:
   1.

   Hint: *Think: What happens when promotions change weekly?*

   2.

   Hint: *Think: Can a fine-tuned model cite specific page numbers?*

4. Write the system prompt for their RAG-powered customer support bot:

   YOUR SYSTEM PROMPT:

5. A customer asks: "*Can I return my headphones after 45 days?*"
   What would the **Retrieval** step find?
   What would the **Augmented Prompt** look like?

Recap: Your Learning Map

- **The Problem:** LLMs are frozen in time, hallucinate, and can't access private data.
- **RAG** solves this by feeding the AI real documents before it answers (open-book exam).
- **The Pipeline:** User Query → Retrieval → Augmentation → Generation.
- **Embeddings** convert text into vectors that capture *meaning*, not just characters.
- **Semantic Search** finds "Automobile" when you search for "Car" — keyword search can't.
- **Vector Databases** (ChromaDB, FAISS, Pinecone) store and search embeddings at scale.
- **RAG = Knowledge** (facts, data). **Fine-Tuning = Behavior** (style, tone).

- **State-of-the-Art:** The best systems combine RAG + Fine-Tuning + Prompt Engineering.